

Comprehensive Technical Analysis of I2C Addressing, Register Configuration, and Communication Architectures for the Arducam OV9281 Global Shutter Image Sensor

Executive Summary

The Arducam OV9281 is a 1-megapixel (1280x800) monochrome global shutter image sensor that has become a cornerstone component in high-speed computer vision applications, including simultaneous localization and mapping (SLAM), drone collision avoidance, and industrial barcode scanning. While the high-bandwidth image data is transmitted via the MIPI CSI-2 interface, the command, control, and telemetry of the sensor are governed entirely by the Serial Camera Control Bus (SCCB), a proprietary protocol by OmniVision that is functionally compatible with the industry-standard Inter-Integrated Circuit (I2C) protocol.

The successful integration of the OV9281 into embedded ecosystems—specifically NVIDIA Jetson and Raspberry Pi platforms—relies heavily on a nuanced understanding of its addressing architecture. Unlike simple peripheral sensors with static addresses, the OV9281 features a dynamic, hardware-strap-controlled addressing scheme complemented by a software-programmable override capability. This report provides an exhaustive technical dissection of these communication mechanisms. It delineates the specific I2C address maps, analyzes the electrical characteristics of the SID (Slave ID) configuration pin, and explores the kernel-level driver structures required to manage multi-camera arrays without bus contention. Furthermore, it provides detailed strategies for resolving common integration challenges, such as signal integrity issues on shared buses and race conditions during initialization sequences.

1. Architectural Overview of the OV9281 Control Plane

The OmniVision OV9281 sensor utilizes a split-interface architecture typical of high-performance CMOS imaging sensors (CIS). While pixel data flows over the high-speed differential pairs of the MIPI D-PHY, the sensor's "brain"—its internal register bank—is accessed via a slower, two-wire serial interface.³ This control plane is responsible for every aspect of sensor operation prior to and during image streaming, including the initialization of the analog front end (AFE), the setting of integration times (exposure), the adjustment of

analog and digital gains, and the management of frame synchronization triggers.⁵

1.1 The Serial Camera Control Bus (SCCB)

Although commonly referred to as I2C in Linux driver documentation and community forums, the OV9281 technically utilizes the Serial Camera Control Bus (SCCB). For all practical engineering purposes in the context of modern embedded Linux systems (like the Jetson Nano or Raspberry Pi), SCCB is treated as standard I2C. The physical layer consists of two lines:

- **SIO_C (Serial Interface Clock):** Functionally equivalent to I2C SCL.⁵
- **SIO_D (Serial Interface Data):** Functionally equivalent to I2C SDA.⁵

The operational frequency typically defaults to standard mode (100 kHz) or fast mode (400 kHz).⁶ Understanding this equivalence is crucial because it allows the use of standard Linux I2C tools (like `i2c-tools`) and standard kernel subsystems (`drivers/media/i2c`) to interface with the device without requiring custom bit-banging drivers.⁷

1.2 The Criticality of Addressing in Machine Vision

In simple consumer applications, a camera is often a solitary device on its bus. However, the OV9281 is predominantly used in machine vision contexts requiring depth perception or broad field-of-view coverage, necessitating stereo (2-camera) or quad-camera setups.⁸ Here, the addressing architecture becomes the primary system bottleneck. If two identical sensors are placed on the same I2C bus with the same address, bus contention occurs: both devices attempt to acknowledge (ACK) commands simultaneously, or worse, one drives the data line low while the other attempts to drive it high, leading to undefined behavior and communication failure.⁹

The OV9281 addresses this challenge through a hierarchical addressing strategy involving:

1. **Hardware Selection:** Using the SID pin to toggle between two hardcoded addresses.
2. **Software Reassignment:** Using internal registers to reprogram the sensor's responsiveness to a third, arbitrary address.

2. Comprehensive I2C Address Specification

The following table serves as the definitive reference for communicating with the OV9281 module. It distinguishes between the 7-bit notation used by Linux/Unix systems and the 8-bit notation used in hardware datasheets—a frequent source of implementation errors.

Table 1: Master I2C Address Map for OV9281

Address Mode	SID Pin State	7-Bit Address (Linux / Arduino)	8-Bit Write Address (Datasheet)	8-Bit Read Address (Datasheet)	Driver Hex Definition	Usage Context
Primary Default	Logic LOW (GND)	0x60	0xC0	0xC1	0x60	The standard address for a single camera or the active camera during initialization. Default state for B0223 modules. ¹⁰
Secondary Default	Logic HIGH (DOVDD)	0x10	0x20	0x21	0x10	Used as a "parking" or "masked" address. Sensors are moved here to effectively silence them while another sensor is configured. ¹⁰

Program mable Base	Indeterm inate	0x70	0xE0	0xE1	0x70	The default target address for reprogra mming the SCCB ID register (0x302B) ¹⁰
General Call	N/A	0x00	0x00	N/A	0x00	Reserved I2C General Call. Generally avoided in camera drivers to prevent resetting other periphera ls. ¹¹

2.1 Insights into the Addressing Logic

2.1.1 The "0x60" Standard

The address 0x60 is the canonical identity of the OV9281. When a developer connects a single Arducam B0223 module to a Raspberry Pi and runs `i2cdetect -y 1`, the expected output is 60 (if the driver is not loaded) or UU (if the driver is loaded).⁷ This dominance of 0x60 stems from the internal pull-down resistor on the module's PCB, which biases the SID pin to Ground by default.¹² This design choice simplifies single-camera integration, allowing "plug-and-play" behavior without requiring GPIO manipulation.

2.1.2 The "0x10" Strategy

The secondary address 0x10 is rarely used as an active communication channel for streaming. Instead, structurally, it functions as a holding pen. In driver source code, specifically `ov9281.c` in the NVIDIA Jetson kernel, initialization routines often begin by driving

the SID pins of *all* connected cameras High.¹⁰ This forces all cameras to the 0x10 address. The system then drops the SID pin of the *first* camera Low, moving it to 0x60. It is then configured and potentially assigned a new software address (e.g., 0x36). Once distinct, the second camera's SID is dropped Low, moving it to 0x60, where it can be configured without conflict because the first camera is now responding to 0x36 (or 0x10). This sophisticated dance relies entirely on the electrical characteristics of the SID pin.

2.1.3 The 7-bit vs. 8-bit Ambiguity

The distinction between 7-bit and 8-bit addressing is a critical technical nuance. The I2C protocol specifies a 7-bit address field. However, the physical byte transmitted on the wire includes the Read/Write bit at the Least Significant Bit (LSB) position.

- **Equation:** $8_Bit_Address = (7_Bit_Address \ll 1) + R/W_Bit$
- **Scenario:** A datasheet might specify the Write Address as 0xC0.
 - Binary: 1100 0000
 - Shift Right 1: 0110 0000 -> 0x60
- **Implication:** Linux tools (i2c-tools, device-tree) and libraries (Python smbus) exclusively use the 7-bit notation (0x60). Using 0xC0 in a Linux device tree will result in the kernel attempting to communicate with address 0x60 shifted left again, effectively 0x80, which is out of range, or simply failing to probe.¹¹

3. Hardware Interface and Electrical Specifications

To communicate reliably with the OV9281, the physical layer must be robust. This involves the mating connectors, pin definitions, and voltage levels, which vary slightly between the bare sensor and the Arducam breakout modules.

3.1 Connector Definition

The raw OV9281 sensor module (as found on the Arducam breakout boards) utilizes a specific mating connector.

- **Part Number:** 51049-03001-001
- **Manufacturer:** Aces Electronics³
- **Type:** 30-pin, high-density FPC connector.

3.2 Pin Definition and Signal Roles

The table below details the specific pinout related to power and control, extracted from the Arducam datasheet.³ Note that while the connector has 30 pins, only a subset is relevant for I2C communication.

Table 2: Critical Control and Power Pins (ACES 30-Pin)

Pin No.	Pin Name	Type	Description	Function in I2C Context
7	SIOC	Input	Serial Interface Clock	Connects to host SCL. Requires pull-up (typically 2.2k Ω - 4.7k Ω). ¹³
8	SIOD	I/O	Serial Interface Data	Connects to host SDA. Bidirectional; requires pull-up. ¹³
9	XSHUTDOWN	Input	Reset / Power Down	Active Low. Must be driven High to enable I2C communication. If Low, sensor is in hardware standby and will not ACK I2C commands. ¹⁴
10	FSIN	I/O	Frame Sync Input	Used for hardware synchronization. Can be configured via register 0x3006. ⁶
25	XCLK	Input	External Clock	Input clock (typically 24MHz). The I2C block

				requires this clock to be stable before register operations can succeed (depending on internal PLL settings). ¹⁵
28	DOVDD	Power	Digital I/O Voltage	1.8V nominal. Defines the logic high level for I2C signals. ¹⁴
-	SID	Input	Slave ID Select	(Often mapped to specific pins or internal pads depending on module revision). Controls the 0x60/0x10 address selection. ¹⁰

3.3 Electrical Considerations for I2C

- Voltage Levels:** The OV9281 sensor core is natively a 1.8V device (DOVDD).¹⁴ However, Arducam modules (e.g., B0223 for Jetson) often include onboard level shifters (LDOs and MOSFETs) to make the interface compatible with 3.3V hosts like the Raspberry Pi and Jetson Nano.¹²
 - Risk:** Connecting a raw sensor (without the Arducam PCB) directly to a 3.3V I2C bus will damage the sensor's I/O pads.
- Pull-Up Resistors:** The I2C bus is an open-drain architecture. The Jetson Nano carrier board typically includes 2.2k Ω pull-ups on the module side and 1k Ω pull-ups on the carrier side.¹³ Missing pull-ups are a common cause of detection failure.
- Power Sequencing:** The datasheet mandates a specific power-up sequence. DOVDD (1.8V) must be stable, and XCLK must be running *before* the XSHUTDOWN pin is pulled high. Only after XSHUTDOWN is high can I2C communication begin. Violating this

sequence often results in the sensor failing to ACK at address 0x60.¹⁴

4. Register-Level Configuration and Reprogramming Logic

The OV9281 provides a "Programmable SCCB ID" feature, which allows the host to redefine the sensor's address. This is a volatile setting; it resets to the hardware default (based on SID) upon power cycling.

4.1 Key Configuration Registers

Based on the driver analysis ¹⁰, the following registers are pivotal for communication management.

Table 3: I2C and System Control Registers

Register Address (Hex)	Register Name	Default Value	Description
0x302B	SC_CTRL_SCCB_ID_ADDR	0xE0	The Programmable Address Register. The value written here determines the <i>new</i> 8-bit Write Address the sensor will respond to. ¹⁰
0x303B	SC_CTRL_3B	-	Bit ¹ (SCCB_PGM_ID_EN) usually needs to be set to '1' to enable the use of the ID programmed in 0x302B. ¹⁰
0x300A	SC_CHIP_ID_HIGH	0x92	Chip ID High Byte. Read-only. Used to verify connection. ¹⁰

0x300B	SC_CHIP_ID_LOW	0x81	Chip ID Low Byte. Read-only. Combined with High Byte = 0x9281. ¹⁰
0x0100	SC_MODE_SELECT	0x00	Mode Control. 0x00 = Standby, 0x01 = Streaming. I2C remains active in Standby. ⁴
0x0103	SC_SOFT_RESET	0x00	Writing 0x01 triggers a software reset, reverting the I2C address to the hardware default (0x60 or 0x10). ¹⁰

4.2 Algorithm for Software Address Reprogramming

To utilize two cameras on the same bus without a multiplexer, one must be moved to a new address. The following algorithm, derived from the `ov9281_i2c_addr_assign` kernel function ¹⁰, achieves this:

1. **Preparation:** Assert XSHUTDOWN (Low) on both cameras to reset them. Release XSHUTDOWN (High) to boot.
2. **Isolation:** Set the SID pin **High** for Camera A and **High** for Camera B. Both cameras are now listening at 0x10.
3. **Target Selection:** Set the SID pin **Low** for Camera A. Camera A moves to 0x60. Camera B stays at 0x10.
4. **Reprogramming:**
 - Send I2C Write to Address 0x60 (Camera A).
 - Register: 0x302B.
 - Value: 0x6C (This is the 8-bit representation of the desired 7-bit address 0x36).
 - *Note:* The value written must be the 8-bit Write address (`Target_7bit << 1`).
5. **Activation:** Write to Register 0x303B to enable the programmable ID if necessary.
6. **Release:** Set SID pin **High** for Camera A.
 - *Insight:* Crucially, once programmed, the sensor tracks the custom address. Even if SID is toggled, it may retain the programmed ID in the "programmable slot" (effectively replacing 0x70 with 0x36), or it might treat the programmed ID as the new primary. Driver analysis suggests the sensor responds to the programmed address regardless of SID state once configured, or requires SID logic to manage the *default* slots while the *programmed* slot remains active.

7. **Enumeration:** Now, Camera A responds to 0x36. Camera B (with SID High) responds to 0x10. You can now pull Camera B's SID Low to make it respond to 0x60.
 - **Result:** Camera A at 0x36, Camera B at 0x60. Bus conflict resolved.
-

5. Platform-Specific Implementation Strategies

The integration of the OV9281 varies significantly between the NVIDIA Jetson and Raspberry Pi ecosystems due to differences in the underlying Linux kernel drivers and hardware topologies.

5.1 NVIDIA Jetson (Nano / Xavier NX / Orin)

The Jetson platform uses the V4L2 (Video4Linux2) framework and the Tegra VI (Video Input) subsystem.

5.1.1 I2C Bus Mapping

On the Jetson Nano B01 and Orin Nano, the camera I2C lines are often routed through an internal I2C Mux (e.g., TCA9548 or internal Tegra pinmuxing).¹⁶

- **Bus Enumeration:** Users often find the cameras on i2c-6, i2c-7, i2c-9, or i2c-10 depending on the specific carrier board and Device Tree Overlay applied.¹⁸
- **Command:** `sudo i2cdetect -y -r 6` is the standard diagnostic command.
- **"UU" vs "60":**
 - **60:** The device is present, but the driver has *not* loaded. This indicates an electrically correct connection but a software/Device Tree mismatch.
 - **UU:** The device is present and the driver *has* loaded (Kernel ownership). This is the desired state for operation.⁷

5.1.2 Driver Source Code Analysis (ov9281.c)

The source code provided in the snippets¹⁰ reveals that the driver explicitly manages the `cam_sid_gpio`.

- **Power On Sequence:** The driver's `ov9281_power_on` function toggles the GPIOs to ensure the correct startup state.
- **Address Assign:** The `ov9281_i2c_addr_assign` function contains the logic described in Section 4.2.
- **Device Tree:** The `.dtsi` file for the Jetson must define the `cam_sid_gpio` property for the driver to utilize the dynamic addressing feature. If this definition is missing, the driver will default to assuming a static address of 0x60 and fail if multiple sensors are present.¹⁰

5.2 Raspberry Pi (Compute Module and Standard Boards)

The Raspberry Pi integration relies heavily on the dtoverlay mechanism in /boot/config.txt.

5.2.1 Single Camera

Adding dtoverlay=ov9281 usually binds the driver to i2c-10 (the camera I2C bus) at address 0x60.⁹

5.2.2 Stereo Configuration (Compute Module 4)

The Compute Module 4 (CM4) has two CSI ports (CAM0 and CAM1).

- **Independent Buses:** Unlike the Jetson Nano B01 which often shares an I2C bus for both cameras, the CM4 routes CAM0 and CAM1 to separate I2C buses (often i2c-0 and i2c-1 or virtual muxes thereof).⁹
- **Conflict Avoidance:** Because they are on separate electrical buses, both cameras can effectively utilize address 0x60 simultaneously. The kernel distinguishes them by their bus number (e.g., 10-0060 vs 0-0060). This simplifies the addressing requirements compared to the Jetson's shared-bus topology.

5.2.3 Arducam "Camarray" HAT

For standard Raspberry Pis (e.g., Pi 4B) with only one CSI connector, or for quad-camera setups, Arducam utilizes a "Camarray" HAT.

- **Addressing Hiding:** This HAT acts as a bridge. It presents a single I2C address to the Pi (often 0x60 or a custom ID). The HAT itself handles the I2C communication to the 2 or 4 attached OV9281 sensors using an onboard FPGA or microcontroller.⁸
- **Implication:** The user does not need to manage individual sensor addresses; the system sees one giant "virtual" sensor.

6. Multi-Camera Synchronization and Triggering

While I2C handles addressing, the synchronization of multiple global shutter sensors relies on the **FSIN** pin, but I2C plays a configuration role.

6.1 Trigger Register Configuration

To enable external triggering (where one camera or an external signal drives the FSIN pin of all sensors), the I2C registers must be configured to switch the sensor from "Master" (generating frames internally) to "Slave" (waiting for a pulse).

- **Register 0x3006:** Controls the direction of the FSIN pin.
 - Write 0x00: Input mode (Slave).⁶
- **Register 0x3823:**
 - Bit²: ext_vs_en (External VSYNC enable). Set to High for trigger mode.⁶
- **Register 0x3030:** Frame Sync output enable (if configuring one camera as the master to

trigger others).

6.2 The "Master-Slave" Addressing Implication

In a Master-Slave sync setup (one OV9281 triggering another), they still require distinct I2C addresses if they share a bus. The I2C bus configures *Trigger Mode* on the Slave and *Streaming Mode* on the Master. The Master then sends the electrical pulse via FSIN to the Slave.

7. Troubleshooting Common Addressing Issues

7.1 "Remote I/O Error" (NACK)

If i2cdetect is slow or returns errors:

- **Check Voltage:** Ensure the 1.8V/3.3V logic levels are compatible.
- **Check Clock:** The OV9281 requires XCLK (24MHz) to be running to respond to I2C queries in many power states. If the driver hasn't enabled the clock (via the mclk pin on the Jetson/Pi), the sensor I2C state machine may remain dormant.¹⁵

7.2 Detection of 0x20 instead of 0x60

If a single sensor appears at 0x20:

- **Cause:** The SID pin is being held High. This could be a PCB fault, a solder bridge, or an inverted GPIO setting in the device tree (active_low vs active_high).¹⁰

7.3 Intermittent "Flapping" between Addresses

- **Cause:** Floating SID pin. If the pull-down resistor is damaged or missing, the address may drift between 0x60 and 0x10 based on electromagnetic interference.

8. Conclusion

The Arducam OV9281's addressing architecture is a robust, multi-layered system designed to accommodate the complex requirements of embedded machine vision. By offering a primary address (**0x60**), a secondary masking address (**0x10**), and a software-programmable override (**0x70** base, remappable via **0x302B**), the sensor provides developers with the flexibility to construct sophisticated multi-camera arrays.

However, this flexibility imposes a burden of responsibility on the system integrator. Developers must rigorously manage the **SID** pin state via GPIOs during the boot sequence, strictly adhere to the **8-bit vs. 7-bit** addressing nomenclature in their code, and correctly

configure the underlying I2C multiplexers or buses on their carrier boards. Failure to adhere to these protocols results in the classic symptoms of bus contention: missing devices, random data corruption, and initialization failures.

This report confirms that for the vast majority of users, the target address is **0x60**. For advanced users building stereo or quad-camera systems on shared buses, the `ov9281_i2c_addr_assign` logic—toggling SID pins to isolate and reprogram sensors sequentially—is the industry-standard methodology for conflict resolution.

9. Appendix: Reference Data

9.1 Register Summary Table

Register	Name	R/W	Description
0x300A	CHIP_ID_H	R	Chip ID High Byte (0x92)
0x300B	CHIP_ID_L	R	Chip ID Low Byte (0x81)
0x302B	SCCB_ID	W	Programmable SCCB ID (8-bit format)
0x3006	FSIN_CTRL	W	FSIN Pin Direction Control
0x3500-02	EXPO	W	Exposure Control (20-bit)
0x3508-09	GAIN	W	Gain Control (16-bit)
0x3820	VFLIP	W	Vertical Flip Control
0x3821	HMIRROR	W	Horizontal Mirror Control

9.2 Schematic Symbol Pinout (ACES 51049-30)

Pin 1: NC	Pin 16: MDP2 (Lane 2+)
Pin 2: NC	Pin 17: MDN2 (Lane 2-)
Pin 3: DVDD (1.2V)	Pin 18: DGND
Pin 4: AGND	Pin 19: MCP (Clock+)
Pin 5: AVDD (2.8V)	Pin 20: MCN (Clock-)
Pin 6: DGND	Pin 21: DGND
Pin 7: SIOC (SCL)	Pin 22: MDP1 (Lane 1+)
Pin 8: SIOD (SDA)	Pin 23: MDN1 (Lane 1-)
Pin 9: XSHUTDOWN	Pin 24: DGND
Pin 10: FSIN	Pin 25: XCLK (MCLK)
Pin 11: DGND	Pin 26: DGND
Pin 12: DGND	Pin 27: NC
Pin 13: NC	Pin 28: DOVDD (1.8V)
Pin 14: DGND	Pin 29: DGND
Pin 15: DGND	Pin 30: NC

1

Works cited

1. OV9281 MIPI Camera Module - RobotShop, accessed February 8, 2026, <https://cdn.robotshop.com/media/a/adu/rb-adu-115/pdf/arducam-ov9281-1mp-global-shutter-mipi-camera-raspberry-pi-datasheet.pdf>
2. OV9281 – 1.0 MP Monochrome Global Shutter Camera Module - dlscorp, accessed February 8, 2026, <https://dlscorp.com/shop/ov9281-1-0-mp-monochrome-global-shutter-camera-module/>
3. OV9281 MIPI Camera Module - Kamami.pl, accessed February 8, 2026, https://download.kamami.pl/p586972-OV9281_MIPI_Camera_Module_Standalone_DS.pdf
4. OV9281 - OmniVision, accessed February 8, 2026, <https://www.ovt.com/products/ov9281/>
5. OV9281 - OmniVision, accessed February 8, 2026, <https://www.ovt.com/wp-content/uploads/2024/05/OV9281-PB-v1.4-WEB.pdf>
6. OV9281 with external trigger - Raspberry Pi Forums, accessed February 8, 2026, <https://forums.raspberrypi.com/viewtopic.php?t=293959>
7. 07.Jetson Nano B01 I2C Communication Tutorial - Yahboom, accessed February

- 8, 2026,
<https://www.yahboom.net/public/upload/upload-html/1732696364/07.Jetson%20Nano%20B01%20I2C%20Communication%20Tutorial.html>
8. [Tutorial] 4 Sync OV9281 Cameras on 1 Raspberry Pi - YouTube, accessed February 8, 2026, <https://www.youtube.com/watch?v=jW4gcla1aOE>
 9. CSI interface and i2c bus - StereoPi Forum, accessed February 8, 2026, <https://forum.stereopi.com/viewtopic.php?t=831>
 10. ov9281.c - GitHub, accessed February 8, 2026, <https://github.com/antmicro/tx2-deep-learning-kit-bsp/blob/master/kernel/kernel-4.4/drivers/media/i2c/ov9281.c>
 11. 7-bit 8-bit and 10-bit I2C Slave Addressing - Total Phase, accessed February 8, 2026, <https://www.totalphase.com/support/articles/200349176-7-bit-8-bit-and-10-bit-i2c-slave-addressing/>
 12. Arducam B0223 OV9281 1MP Global Shutter NoIR Camera Module for Jetson Nano, accessed February 8, 2026, <https://www.welectron.com/Arducam-B0223-OV9281-1MP-Global-Shutter-NoIR-Camera-Module-for-Jetson-Nano>
 13. Jetson Orin Nano Developer Kit Carrier Board Specification, accessed February 8, 2026, https://developer.nvidia.com/downloads/assets/embedded/secure/jetson/orin_nano/docs/jetson_orin_nano_devkit_carrier_board_specification_sp.pdf
 14. datasheet OV9281 - PI Camera-V-Vision Tech, accessed February 8, 2026, https://www.v-visiontech.com/web/userfiles/download/OV9281_CSP5_DS_1.01.pdf
 15. OV9281 - Raspberry Pi Forums, accessed February 8, 2026, <https://forums.raspberrypi.com/viewtopic.php?t=291818>
 16. 07.Jetson Nano I2C Communication Tutorial - Yahboom, accessed February 8, 2026, <https://www.yahboom.net/public/upload/upload-html/1683344878/07.Jetson%20Nano%20I2C%20Communication%20Tutorial.html>
 17. Use i2c mux on custom carrier board - Jetson Orin Nano - NVIDIA Developer Forums, accessed February 8, 2026, <https://forums.developer.nvidia.com/t/use-i2c-mux-on-custom-carrier-board/330163>
 18. I2C port # for CAM_I2C for Nano? - Jetson Nano - NVIDIA Developer Forums, accessed February 8, 2026, <https://forums.developer.nvidia.com/t/i2c-port-for-cam-i2c-for-nano/167267>
 19. Orin nano J20 and J21 connect a camera respectively, but J21 camera can not working properly, will stall within a few minutes of starting, accessed February 8, 2026, <https://forums.developer.nvidia.com/t/orin-nano-j20-and-j21-connect-a-camera-respectively-but-j21-camera-can-not-working-properly-will-stall-within-a-few-minutes-of-starting/300215>
 20. Arducam cameras fitting Raspberry CSI-2 interface, accessed February 8, 2026,

<https://forums.raspberrypi.com/viewtopic.php?t=249483>